Pi master and ESP PID workflow

This project keeps the ESP32 as the protected motion controller. The Raspberry Pi sends high-level position commands only, then waits for acknowledgements, `done` events, fresh telemetry, or IR edge events before sending the next command.

Files

- `esp_pid.ino`: ESP32 FreeRTOS mecanum controller. It owns wheel PID, heading hold, encoder/IMU odometry, and UART telemetry. Do not edit unless a real ESP-side bug is confirmed.
- `pi_master.py`: Raspberry Pi mission, training, path, and IR debug brain.
- `pi_mecanum_path_builder.py`: Smaller interactive position/path tester.

ESP command contract

The Pi master uses:

- `PING`
- `INIT_IMU`
- `RESET_ODOM`
- `RESET_ENC`
- `MOVE angle=<deg> dist=<cm> heading=<deg> timeout=<ms>`
- `STOP`

It does not send wheel PWM, micro-motion signals, or `TWIST` in the normal modes.

Pi master modes

Run help:

```
python3 pi_master.py --help
```

Mission mode:

```
python3 pi_master.py --mode mission
```

The script prompts for the goal distance in cm unless `--auto-start` or `--no-prompt-goal` is used.

Move mode:

```
python3 pi_master.py --mode move
```

Interactive commands:

```
F 30
R 20
L 20
B 10
FR 25
FL 25
BR 25
BL 25
status
stop
q
```

Path mode:

```
python3 pi_master.py --mode path
```

Enter one waypoint per line using the same direction syntax. Submit a blank line to run the whole path sequentially.

IR monitor:

```
python3 pi_master.py --mode ir-monitor
```

This prints active IR states plus rising and falling edges. It does not need the ESP link.

IR training:

```
python3 pi_master.py --mode ir-train
```

The robot stays still until an IR rising edge is detected. Front/front-left/front-right triggers run the front obstacle sequence. Left/right triggers run the side falling-edge sequence.

Obstacle sequence

Default mission behavior:

1. Send one forward `MOVE` toward the goal.
2. If a front IR triggers, send `STOP` and wait for the ESP stopped `done` event or fresh telemetry.
3. Bias right first unless the right side is blocked, then switch left.
4. Strafe until the matching front diagonal sensor detects:
    - Strafe right watches `front_left`.
    - Strafe left watches `front_right`.
5. Continue the strafe until the diagonal sensor falls.
6. Move forward until the side sensor sees the obstacle and then falls.

7. Move forward 30 cm.
8. Recenter using odometry first, compensating accumulated lateral movement.
9. Continue toward the original goal distance.
10. If forward progress overshoots the goal, move backward to correct.

`front_right` is present in the logic but is not assigned a GPIO pin yet, so it reports `FR=NA`.

Editable parameters

Common tuning flags:

```
--goal-distance 1.20
--front-advance-distance 0.30
--front-strafe-search-distance 1.20
--side-follow-search-distance 3.00
--preferred-first-direction right
--goal-tolerance 0.05
--rejoin-tolerance 0.02
```

IR interpretation:

```
--ir-logic baseline
--ir-logic active-low
--ir-logic active-high
```

`baseline` is the default. It records the startup GPIO state and treats a changed state as detection. If that behaves badly on the real sensors, test `active-low` because the older IR test reported idle as `1` and detection as `0`.

Logs

Each run creates:

```
run_logs/<timestamp>_<mode>/events.jsonl
run_logs/<timestamp>_<mode>/moves.csv
```

Use `--no-log` to disable logging or `--log-dir <path>` to change the folder.

Safe test order

1. `python3 pi_master.py --mode ir-monitor`
2. `python3 pi_master.py --mode move`
3. Test `F/R/L/B 20` to confirm mecanum direction signs.
4. `python3 pi_master.py --mode ir-train`
5. `python3 pi_master.py --mode mission`